

PC40 - Lab Session 1: Multithreading and Multiprocessing

Language: C++ (C++17, C++20 for exercises using `std::barrier`)

Machine 1: Fedora Linux, GCC 16.2.1, 24 logical processors, used for exercises 1, 2, 3, 4, 6, 8, 9

Machine 2: WSL2 Ubuntu, Intel Core i3-10110U, 4 logical processors, used for exercise 7 (matrix multiplication)

Exercise numbers follow the lab handout (1 to 9). Exercises 10 and 11 (thread pool / process pool for I/O) are not part of this submission.

1) Simple Thread Creation

Code: `ex1/Ex1.cpp`

- a) No.
- b) Maybe, because the threads run concurrently, so interleaving is possible but not guaranteed, especially for a program this short.
- c) Not necessarily, an artificial delay could force interleaving without proving true simultaneous execution.
- d) The threads finish too fast for the scheduler to preempt them, so no context switch happens between `t1` and `t2`.

2) Does Multithreading Always Improve Performance?

Part A, single blocking task, no separate source file.

- a) Both times are nearly identical (about 2.000 s), since in both cases `main` waits for the full 2 s task before measuring the end.
- b) No, `join()` is called right after `start()`, so `main` waits for the thread to finish anyway, no benefit is gained.
- c) `join()` blocks `main` until the thread finishes, making the total elapsed time effectively sequential.
- d) Thread creation, stack allocation, OS scheduling and context switching, and `join()` synchronization overhead, so the threaded version is slightly slower than 2 s rather than faster.

Part B, 50 tasks split across worker threads. Code: `ex3/`

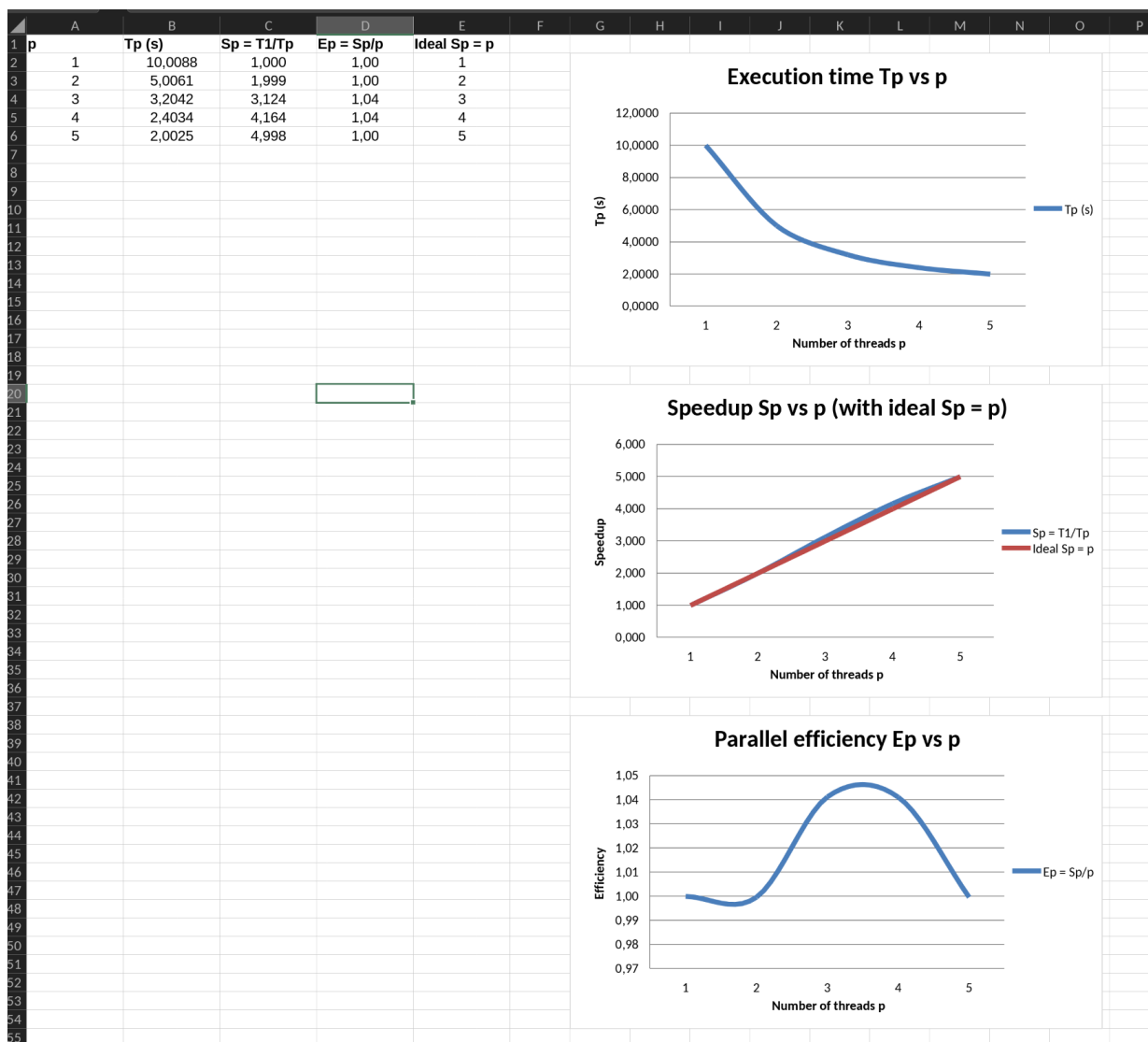
`MultithreadingExampleWorkers.cpp`

- a) $S_p = T_1/T_p$, $E_p = S_p/p$. Speedup is roughly linear (S_p is about p), efficiency close to 100 percent.
- b) The 50 tasks are split across p threads sleeping concurrently, so total time is about $(50/p)$ times 200 ms instead of 50 times 200 ms sequential.
- c) No, `sleep_for` uses no CPU, it is just waiting. The gain comes from concurrency during the wait, not extra computing power.

d) With CPU-bound work, speedup would be limited by the number of physical cores: near-linear up to p equal to the core count, then flattening or degrading beyond it due to core contention and context-switch overhead.

Part C, performance curves.

p	Tp (s)	Sp	Ep
1	10.0088	1.000	1.00
2	5.0061	1.999	1.00
3	3.2042	3.124	1.04
4	2.4034	4.164	1.04
5	2.0025	4.998	1.00



Tp, Sp (with ideal Sp=p) and Ep versus number of threads p

Speedup stays essentially linear and efficiency stays close to 1.00 throughout, expected here since the workload is pure waiting with no CPU contention.

3) Deadlock Simulation and Prevention

Code (corrected version, consistent lock ordering): `ex5/deadlock.cpp`

- a) It occurred because t1 held mu1 while waiting for mu2, and t2 held mu2 while waiting for mu1, so each thread was blocked waiting for a lock the other one held.
- b) All four Coffman conditions held: mutual exclusion, hold-and-wait, no preemption, and circular wait between the two threads.
- c) The deadlock was prevented by enforcing a consistent global lock order, so both threads always acquire mu1 before mu2.
- d) The trade-off is reduced concurrency, since the two threads now execute their critical sections more sequentially instead of in parallel.

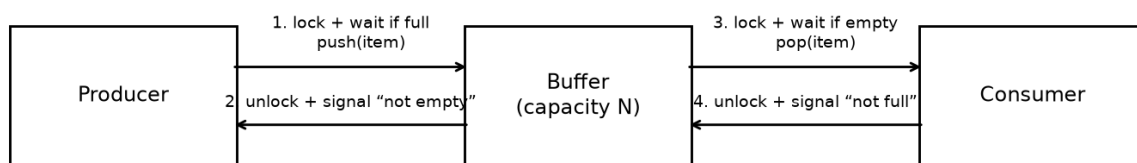
4) Race Condition and Synchronization Using a Mutex

Code: `ex6/raceCondition.cpp`

- a) Theoretically, a final value of 0 was expected.
- b) No, never, without synchronization the final value is not reliably 0.
- c) `i++` and `i--` are read-modify-write operations, not atomic, so one thread's write can overwrite the other's, losing an update.
- d) The mutex protects the operation and avoids the race condition.
- e) Mutex locking forces threads to wait for each other, serializing access and adding overhead, which slows things down compared to the unsynchronized (but incorrect) version.

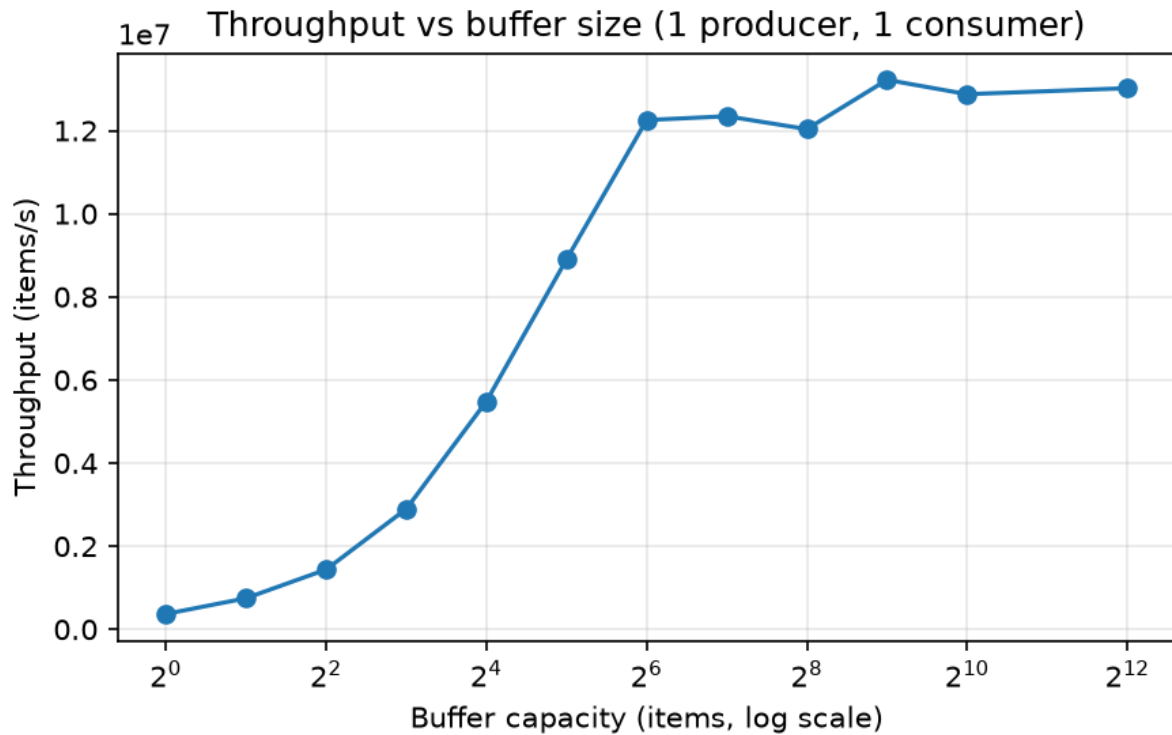
5) Producer-Consumer Problem

Code: `ex7/producer_consumer.cpp`

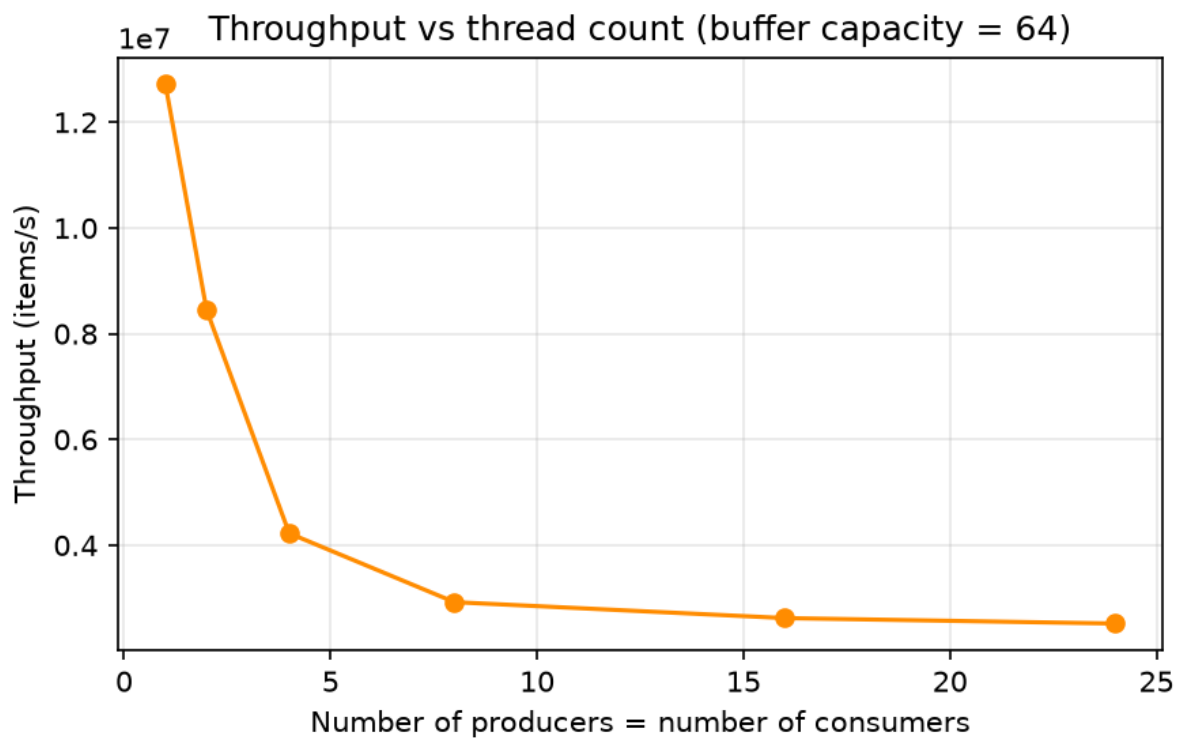


loop until all items are produced / consumed

Producer-consumer execution diagram



Throughput versus buffer capacity



Throughput versus number of producers/consumers

a) Throughput rises with buffer size then saturates around capacity 64, because a bigger buffer lets producer and consumer absorb speed differences instead of blocking on every item.

b) Throughput drops as more producers/consumers are added, because they all share one mutex, so extra threads just add lock contention instead of real parallelism.

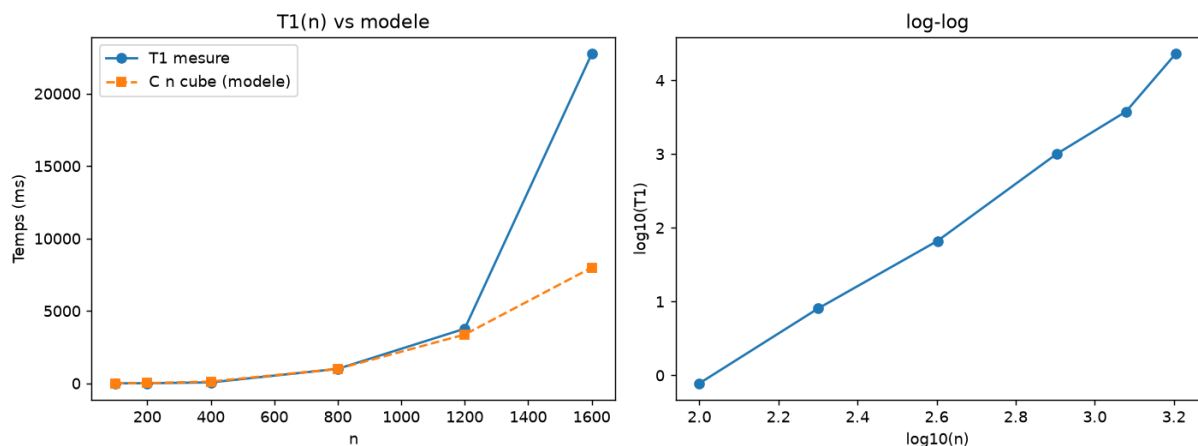
6) Basic Multiprocessing Task

Code: ex8/Multiprocessing.cpp, factorials computed by child processes, results returned to the parent through pipes.

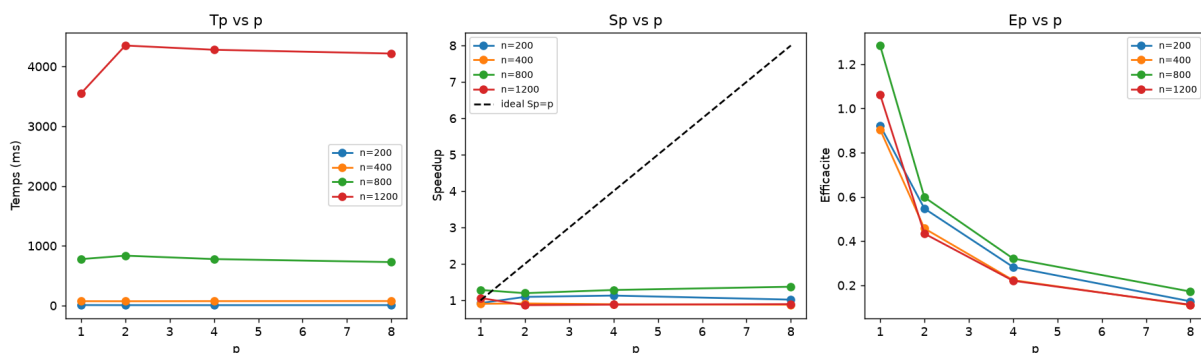
- a) Threads share memory, processes created with fork() each get their own separate copy.
- b) All of it, each child has its own private copy of every variable.
- c) Through a pipe, written by the child and read by the parent.
- d) fork() copies the whole process, and pipe I/O adds kernel calls, both cost more than threads.

7) Matrix Multiplication Using Multiprocessing

Code: ex9/MatrixMultiplication.cpp (sequential), ex9/MatrixMultiplicationProcess.cpp (multiprocessing, one process per row). Full data and formulas in ex9/Ex9_Benchmarks.xlsx.



T1(n) measured versus the normalized $C n^3$ model, and $\log_{10}(T1)$ versus $\log_{10}(n)$



Tp , Sp (with ideal $Sp=p$) and Ep versus number of processes p , for several matrix sizes

- a) T1, Tp, Sp and Ep were measured for n from 100 to 1600 (sequential) and n in {200, 400, 800, 1200} times p in {1, 2, 4, 8} (parallel).
- b) No, Sp stayed between 0.9 and 1.3 across all tested sizes, and Tp sometimes went up when p increased, for example n=1200 with p=2 was slower than p=1.
- c) 4 logical processors (Intel Core i3-10110U, 2 physical cores with hyper-threading).
- d) fork() process-creation cost, serializing the whole result block through the pipe with write/read system calls, and contention between processes for the same 2 physical cores and shared cache.
- e) A small matrix fits entirely in cache and finishes almost instantly, so the fixed cost of fork, pipe and scheduling dwarfs the actual computation, splitting it across processes only adds overhead.
- f) Growth matches n cubed for mid-range sizes, R(200 to 400) is 8.2, close to the theoretical 2 cubed which is 8, but becomes much steeper for large n, R(800 to 1600) is 22.8, once the matrices stop fitting in cache.
- g) Sp stays far below p because fork/pipe overhead grows with p while only 2 physical cores are available, so the fixed cost dominates any gain from splitting the O(n cubed) work, Ep drops to about 0.11 at p=8.

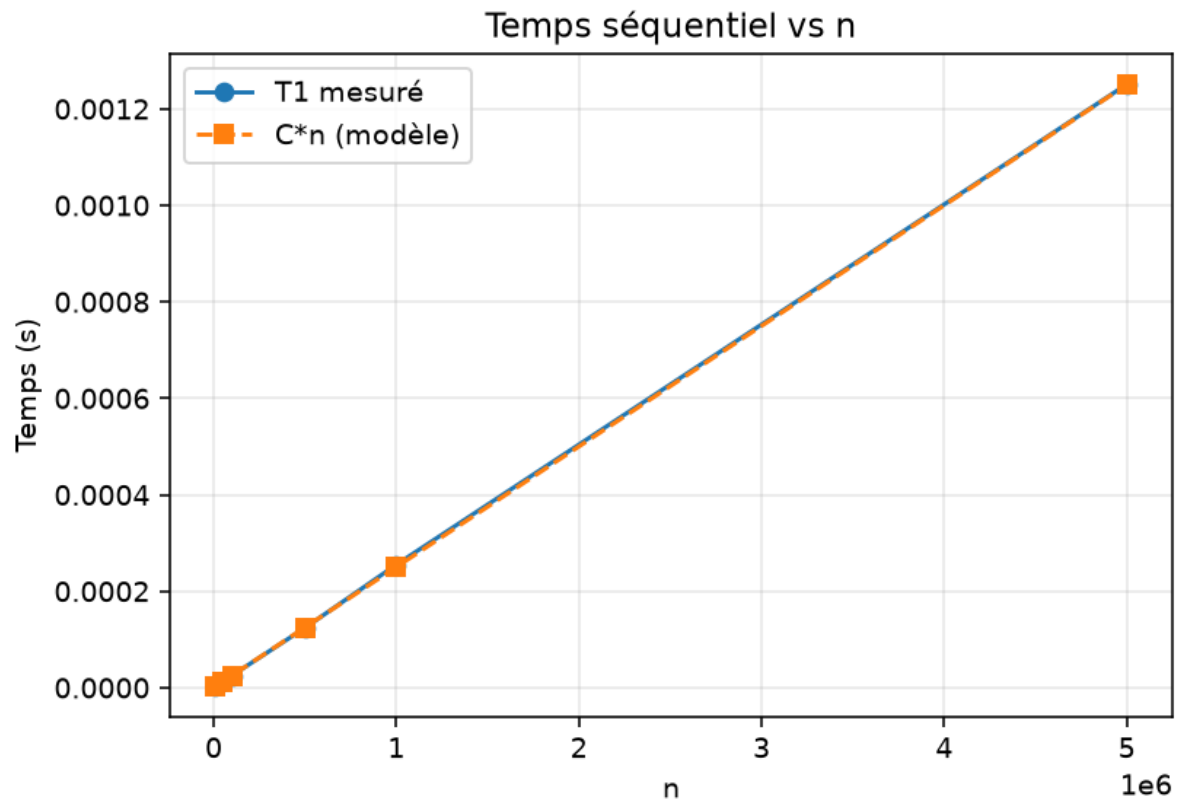
8) Barrier Synchronization

Code: `ex10/BarrierSynchronization.cpp`, using `std::barrier` with 5 threads and staggered phase-1 durations to make the synchronization point observable.

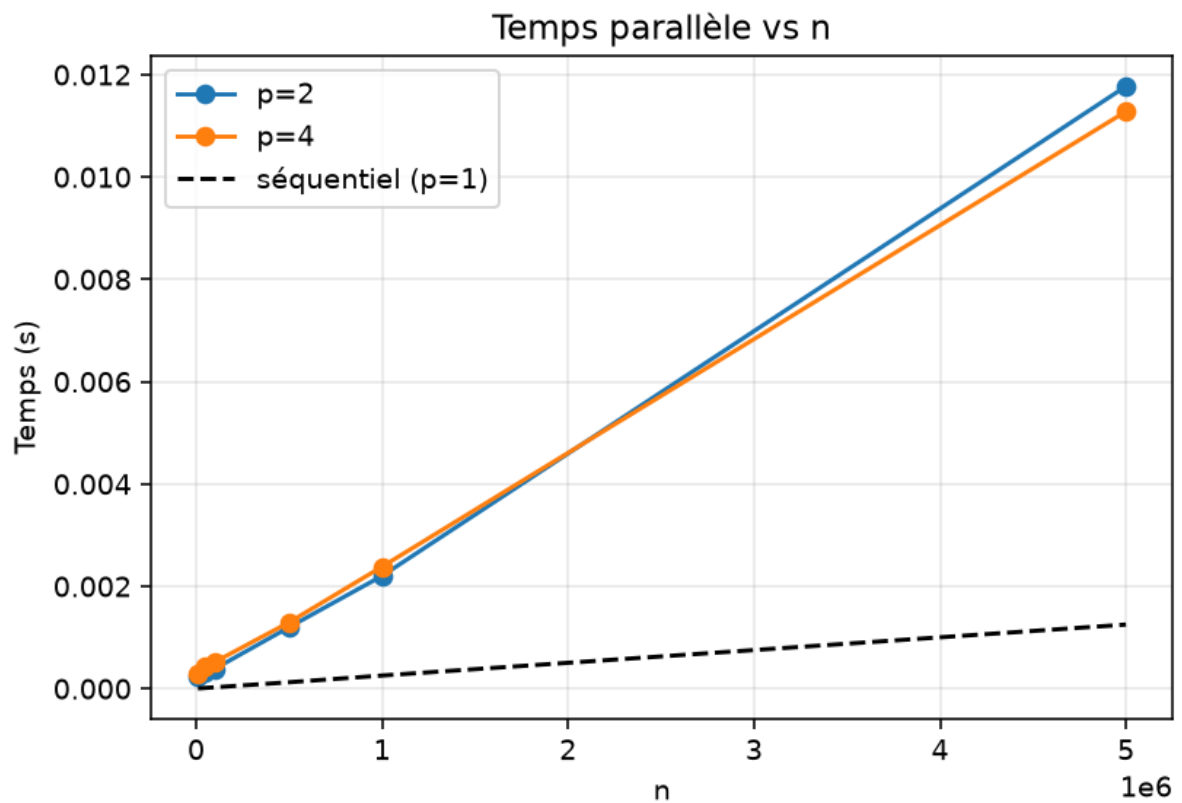
- a) A mutex lets one thread through a section at a time, a barrier lets all threads through, but only once every one of them has arrived.
- b) Example: a physics or numerical simulation running in timesteps, where every thread must finish computing the current timestep before any of them starts the next one.

9) Parallel Map-Reduce

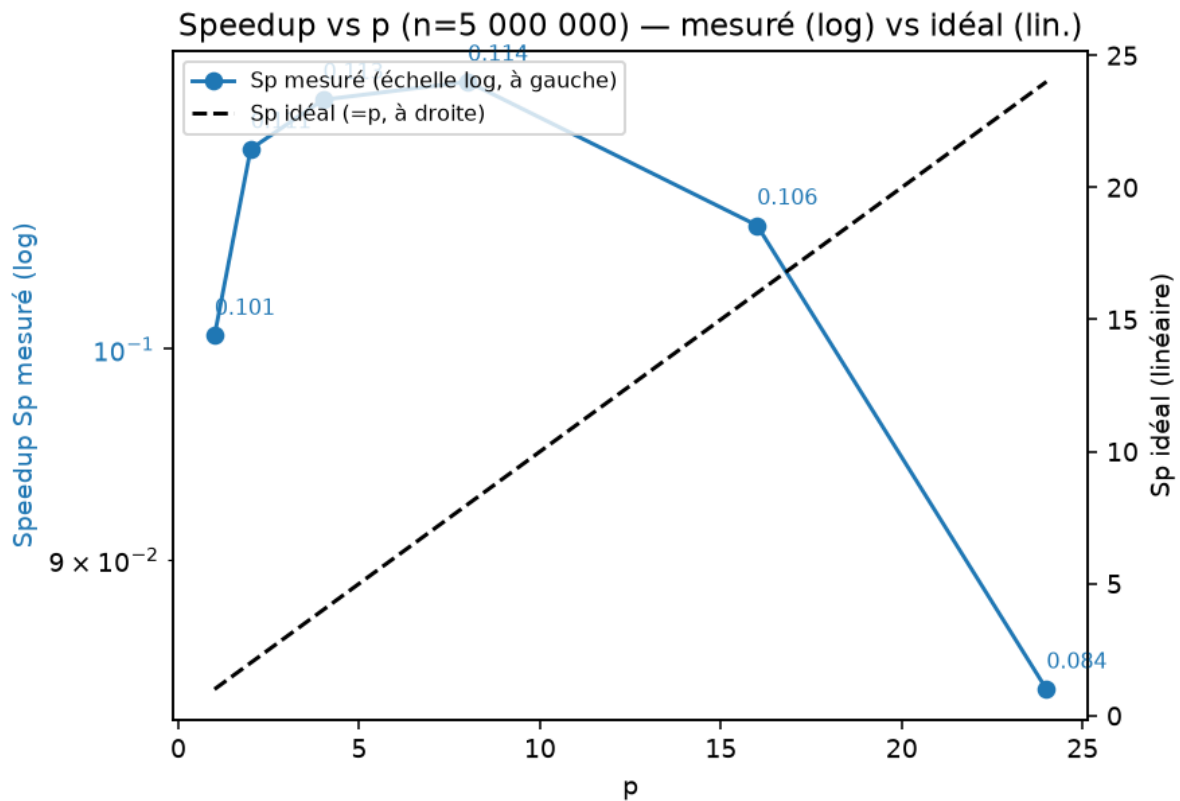
Code: `ex11/MapReduce.cpp` and `ex11/MapReduceFixedWorkers.cpp` (fixed-size worker pool, block-based split of the input array).



Sequential execution time versus n, with normalized C n model



Parallel execution time versus n, for $p=2$ and $p=4$



Speedup Sp versus p, for n=5,000,000

- Sequential growth is consistent with $\Theta(n)$, because doubling n roughly doubles the time.
- No beneficial range for multiprocessing was found here, because overhead never gets amortized.
- The parallel version is slower because the fixed cost of a process and its pipe outweighs such cheap work.
- Speedup versus p stays well below 1, because overhead grows faster than useful work as p increases.

Conclusion

Concurrency structures a program as independent tasks, even on a single core. Parallelism is their actual simultaneous execution on multiple physical cores, and only concurrency built for it benefits, as seen with matrix rows or map-reduce blocks, while a single sleeping thread in exercise 2 only overlapped waiting time. Synchronization (mutex, condition variable, barrier, pipe) coordinates these tasks safely, whenever shared state existed (exercise 4) or coordination points were needed (exercise 8, and pipes in exercises 6, 7, 9), and it always has a cost, sometimes more than the work it protects, as seen when process and pipe overhead outweighed a trivial computation in the map-reduce exercise.